Doc. no.:	P0792R14
Date:	2022-2-8
Audience:	LWG
Reply-to:	Vittorio Romeo <vittorio.romeo@outlook.com> Zhihao Yuan <zy@miator.net> Jarrad Waterloo <descender76@gmail.com>

`function_ref`: a type-erased callable reference

Table of contents

Changelog

R14

- Editorial tweaks

R13

- Refine wording per LWG review (2022-12);
- Add `// freestanding` to avoid merge deficiencies.

R12

- Adjust wording per LWG review (2022-11);
- Correct a deduction guide that mishandled pointer to data member.

R11

- Fix oversights and typos in the wording.

R10

- Integrate the `nontype_t` constructors.

R9

- Declare the main template as variadic for future extension;
- Allow declaring a variable of `function_ref constinit`.

R8

- Stop supporting pointer-to-members;
- Prevent assigning from callable objects other than function pointers while retaining copy assignment;
- Consolidate the wording with better terminologies.

R7

- Clarify the proposal to handle function and function pointers in the same way.

R6

- Avoid double-wrapping existing references to callables;

- Reworked the wording to follow the latest standardese;
- Applied changes requested by LWG (2020-07);
- Removed a deduction guide that is incompatible with explicit object parameters.

R5

- Removed “qualifiers” from `operator()` specification (typo);

R4

- Removed `constexpr` due to implementation concerns;
- Explicitly say that the type is trivially copyable;
- Added brief before synopsis;
- Reworded specification following P1369.

R3

- Constructing or assigning from `std::function` no longer has a precondition;
- `function_ref::operator()` is now unconditionally `const` -qualified.

R2

- Made copy constructor and assignment operator `= default` ;
- Added *exposition only* data members.

R1

- Removed empty state, comparisons with `nullptr` , and default constructor;
- Added support for `noexcept` and `const` -qualified function signatures;
- Added deduction guides similar to `std::function` ;
- Added example implementation;
- Added feature test macro;
- Removed `noexcept` from constructor and assignment operator.

Abstract

This paper proposes the addition of `function_ref<R(Args...)>` , a *vocabulary type* with reference semantics for passing entities to call, to the standard library.

Motivating examples

Here's one example use case that benefits from *higher-order functions*: a `retry(n, f)` function that attempts to call `f` up to `n` times synchronously until success. This example might model the real-world scenario of repeatedly querying a flaky web service.

```
using payload = std::optional< /* ... */ >;

// Repeatedly invokes `action` up to `times` repetitions.
// Immediately returns if `action` returns a valid `payload`.
// Returns `std::nullopt` otherwise.
payload retry(size_t times, /* ????? */ action);
```

The passed-in `action` should be a callable entity that takes no arguments and returns a `payload`. Let's see how to implemented `retry` with various techniques.

Using function pointers

```
payload retry(size_t times, payload(*action)())
{
    /* ... */
}
```

- **Advantages**:
    - Easy to implement: no template, nor constraint. The function pointer type has a signature that nails down which functions to pass.
    - Minimal overhead: no allocations, no exceptions, and major calling conventions can pass a function pointer in a register.
- **Drawbacks**:
    - A function is usually not stateful, nor does captureless closure objects. One cannot pass other function objects in C++ this way.

Using a template

```
template<class F>
auto retry(size_t times, F&& action)
requires std::is_invocable_r_v<payload, F>
{
    /* ... */
}
```

- **Advantages**:
  - Support arbitrary function objects, such as closures with captures.
  - Zero-overhead: no allocations, no exceptions, no indirections.
- **Drawbacks**:
  - Harder to implement: users must constrain `action`'s signature.
  - Fail to support separable compilation: the implementation of `retry` must appear in a header file. A slight change at the call site will cause recompilation of the function body.

Using `std::function` or `std::move_only_function`

```
payload retry(size_t times, std::move_only_function<payload()> action)
{
    /* ... */
}
```

- **Advantages**:
  - Take more *callable objects*, from closures to pointer-to-members.
  - Easy to implement: no need to use a template or any explicit constraint. `std::function` and `std::move_only_function` constructor is constrained.
- **Drawbacks**:
  - `std::function` and `std::move_only_function`'s converting constructor[1] require their target objects to be copy-constructible or move-constructible, respectively;

- Comes with potentially significant overhead:
  - The call wrappers start to allocate the target objects when they do not fit in a small buffer, introducing more indirection when calling the objects.
  - No calling conventions can pass these call wrappers in registers.
  - Modern compilers cannot inline these call wrappers, often resulting in inferior codegen then previously mentioned techniques.

One rarely known technique is to pass callable objects to call wrappers via a `std::reference_wrapper`:

```
auto result = retry(3, std::ref(downloader));
```

But users cannot replace the `downloader` in the example with a lambda expression as such an expression is an rvalue. Meanwhile, all the machinery that implements type-erased copying or moving must still be present in the codegen.

Using the proposed `function_ref`

```
payload retry(size_t times, function_ref<payload()> action)
{
    /* ... */
}
```

- **Advantages**:
  - Takes any *callable objects* regardless of whether they are constructible.
  - Easy to implement: no need to use a template or any constraint. `function_ref` is constrained.
  - Clean ownership semantics: `function_ref` has reference semantics as its name suggests.

- Minimal overhead: no allocations, no exceptions, certain calling conventions can pass `function_ref` in registers.
    - Modern compilers can perform tail-call optimization when generating thunks. If the function body is visible, they can deliver optimal codegen, identical to the template solution.

Design considerations

This paper went through LEWG at R5, with a number of consensuses reached and applied to the wording:

1. Do not provide `target()` or `target_type`;
2. Do not provide `operator bool`, default constructor, or comparison with `nullptr`;
3. Provide `R(Args...) noexcept` specializations;
4. Provide `R(Args...) const` specializations;
5. Require the target entity to be *Lvalue-Callable*;
6. Make `operator()` unconditionally `const`;
7. Choose `function_ref` as the right name.

One design question remains not fully understood by many: how should a function pointer initialize `function_ref`?

In a typical scenario, there is no lifetime issue no matter whether the `download` entity below is a function, a function pointer, or a closure:

```
auto result = retry(3, download); // always safe
```

However, even if the users use `function_ref` only as parameters initially, it's not uncommon to evolve the API by grouping parameters into structures,

```
struct retry_options
{
    size_t times;
    function_ref<payload()> action;
    seconds step_back;
};

payload retry(retry_options);

/* ... */

auto result = retry({.times = 3,
                     .action = download,
                     .step_back = 1.5s});
```

and structures start to need constructors or factories to simplify initialization:

```
auto opt = default_strategy();
opt.action = download;
auto result = retry(opt);
```

According to the P0792R5[2] wording, the code has well-defined behavior if `download` is a function. However, one **cannot** write the code as

```
auto opt = default_strategy();
opt.action = &download;
auto result = retry(opt);
```

since this will create a `function_ref` with a dangling object pointer that points to a temporary object – the function pointer.

In other words, the following code also has undefined behavior:

```
auto opt = default_strategy();
opt.action = ssh.get_download_callback(); // a function pointer
auto result = retry(opt);
```

The users have to write the following to get well-defined behavior.

```
auto opt = default_strategy();
opt.action = *ssh.get_download_callback();
auto result = retry(opt);
```

Survey

We collected the following `function_ref` implementations available today:

■ `llvm::function_ref` (https://github.com/llvm/llvm-project/blob/main/llvm/include/llvm/ADT/STLExtras.h) – from LLVM[3]

■ `tl::function_ref` (https://github.com/TartanLlama/function_ref/blob/master/include/tl/function_ref.hpp) – by Sy Brand

■ `folly::FunctionRef` (https://github.com/facebook/folly/blob/main/folly/Function.h) – from Meta

■ `gdb::function_view` (https://github.com/bminor/binutils-gdb/blob/master/gdbsupport/function-view.h) – from GNU

■ `type_safe::function_ref` (https://github.com/foonathan/type_safe/blob/main/include/type_safe/reference.hpp) – by Jonathan Müller[4]

■ `absl::function_ref` (https://github.com/abseil/abseil-cpp/blob/master/absl/functional/function_ref.h) – from Google

They have diverging behaviors when initialized from function pointers:

Behavior A.1: Stores a function pointer if initialized from a function, stores a pointer to function pointer if initialized from a function pointer

Outcome	**Library**

Undefined:
`opt.action = ssh.get_download_callback();`
Well-defined:
`opt.action = download;`
	■ `llvm::function_ref`
	■ `tl::function_ref`

Behavior A.2: Stores a function pointer if initialized from a function or a function pointer

Outcome	Library

Well-defined:
`opt.action = ssh.get_download_callback();`
Well-defined:
`opt.action = download;`
	■ `folly::FunctionRef`
	■ `gdb::function_view`
	■ `type_safe::function_ref`
	■ `absl::function_ref`

P0792R5 wording gives **Behavior A.1**.

A related question is what happens when initialized from pointer-to-members. In the following tables, assume `&Ssh::connect` is a pointer to member function:

Behavior B.1: Stores a pointer to pointer-to-member if initialized from a pointer-to-member

Outcome	Library

Well-defined:
`lib.send_cmd(&Ssh::connect);`
Undefined:
`function_ref<void(Ssh&)> cmd = &Ssh::connect;`
	■ `tl::function_ref`
	■ `folly::FunctionRef`
	■ `absl::function_ref`

Behavior B.2: Only supports callable entities with function call expression

Outcome	Library

Ill-formed:
`lib.send_cmd(&Ssh::connect);`
Ill-formed:
`function_ref<void(Ssh&)> cmd = &Ssh::connect;`
Well-defined:
`lib.send_cmd(std::mem_fn(&Ssh::connect));`
	■ `llvm::function_ref`
	■ `gdb::function_view`
	■ `type_safe::function_ref`

P0792R5-R7 wording gives **Behavior B.1**.

Additional information

P2472 “make `function_ref` more functional” [5] suggests a way to initialize `function_ref` from pointer-to-members without dangling in all contexts:

```
function_ref<void(Ssh&)> cmd = nontype<&Ssh::connect>;
```

Not convertible from pointer-to-members means that `function_ref` does not need to use `invoke_r` in the implementation, improving debug codegen in specific toolchains with little effort.

Making `function_ref` large enough to fit a thunk pointer plus any pointer-to-member-function may render `std::function_ref` irrelevant in the real world. Some platform ABIs can pass a trivially copyable type of a 2-word size in registers and cannot do the same to a bigger type. Here is some LLVM IR to show the difference: https://godbolt.org/z/Ke3475vz8 (https://godbolt.org/z/Ke3475vz8).

For good or bad, the expression `&qualified-id` that retrieves pointer-to-member shares grammar with the expression that gets a pointer to explicit object member functions. [expr.unary.op/3] (https://eel.is/c++draft/expr.unary.op#3)

Design decisions

- **Behavior A.2** is incorporated to eliminate the difference between initializing `function_ref` from a function and initializing `function_ref` from a function pointer.

- **Behavior B.2** is incorporated to reduce potential damage at a small cost. This change will reflect on the constructor's constraints.

- LEWG further requested making converting assignment from anything other than functions and function pointers ill-formed. Note that `function_ref` will still be copy-assignable.

- LEWG achieved consensus on P2472R3, and the wording is merged here. This change brings back the pointer-to-member support in a different form.

Wording

The wording is relative to N4917 (https://wg21.link/N4917).

Add new templates to [utility.syn] (https://eel.is/c++draft/utility.syn), header `<utility>` synopsis, after `in_place_index_t` and `in_place_index`:

```
// all freestanding
#include <compare>              // see [compare.syn]
#include <initializer_list>     // see [initializer.list.syn]

namespace std {
  [...]

  // nontype argument tag
  template<auto V>
    struct nontype_t {
      explicit nontype_t() = default;
    };

  template<auto V> constexpr nontype_t<V> nontype{};
}
```

Add the template to [functional.syn] (https://eel.is/c++draft/functional.syn), header `<functional>` synopsis:

> [...]

```
  // [func.wrap.move], move only wrapper
  template<class... S> class move_only_function;        // not defined
  template<class R, class... ArgTypes>
    class move_only_function<R(ArgTypes...) cv ref noexcept(noex)>; // see below

  // [func.wrap.ref], non-owning wrapper
  template<class... S> class function_ref;              // freestanding, not defin
  template<class R, class... ArgTypes>
    class function_ref<R(ArgTypes...) cv noexcept(noex)>;           // freestandin
```

[…]

Create a new section “Non-owning wrapper” [func.wrap.ref] with the following after [func.wrap.move] (https://eel.is/c++draft/func.wrap.move):

General

[func.wrap.ref.general]

The header provides partial specializations of `function_ref` for each combination of the possible replacements of the placeholders *cv* and *noex* where:

- *cv* is either `const` or empty.
- *noex* is either `true` or `false`.

Class template `function_ref`

[func.wrap.ref.class]

```
namespace std
{
  template<class... S> class function_ref;    // not defined

  template<class R, class... ArgTypes>
  class function_ref<R(ArgTypes...) cv noexcept(noex)>
  {
  public:
    // [func.wrap.ref.ctor], constructors and assignment operators
    template<class F> function_ref(F*) noexcept;
    template<class F> constexpr function_ref(F&&) noexcept;
    template<auto f> constexpr function_ref(nontype_t<f>) noexcept;
    template<auto f, class U>
      constexpr function_ref(nontype_t<f>, U&&) noexcept;
    template<auto f, class T>
      constexpr function_ref(nontype_t<f>, cv T*) noexcept;

    constexpr function_ref(const function_ref&) noexcept = default;
    constexpr function_ref& operator=(const function_ref&) noexcept = default;
    template<class T> function_ref& operator=(T) = delete;

    // [func.wrap.ref.inv], invocation
    R operator()(ArgTypes...) const noexcept(noex);
  private:
    template<class... T>
      static constexpr bool is-invocable-using = see below;   // exposition only
  };

  // [func.wrap.ref.deduct], deduction guides
  template<class F>
    function_ref(F*) -> function_ref<F>;
  template<auto f>
    function_ref(nontype_t<f>) -> function_ref<see below>;
  template<auto f>
    function_ref(nontype_t<f>, auto) -> function_ref<see below>;
}
```

An object of class `function_ref<R(Args...) cv noexcept(noex)>` stores a pointer to function *`thunk-ptr`* and an object *`bound-entity`*. *`bound-entity`* has an unspecified trivially copyable type `BoundEntityType`, that models `copyable` and is capable of storing a pointer to object value or a pointer to function value. The type of *`thunk-ptr`* is `R(*)(BoundEntityType, Args&&...) noexcept(noex)`.

Each specialization of `function_ref` is a trivially copyable type [basic.types] (https://eel.is/c++draft/basic.types) that models `copyable`.

Within this subclause, *`call-args`* is an argument pack with elements such that `decltype((call-args))...` denote `Args&&...` respectively.

Constructors and assignment operators

[func.wrap.ref.ctor]

```
template<class... T>
  static constexpr bool is-invocable-using = see below;
```

If *noex* is true, *`is-invocable-using`*`<T...>` is equal to:

```
  is_nothrow_invocable_r_v<R, T..., ArgTypes...>
```

Otherwise, *`is-invocable-using`*`<T...>` is equal to:

```
  is_invocable_r_v<R, T..., ArgTypes...>
```

```
template<class F> function_ref(F* f) noexcept;
```

Constraints:

- `is_function_v<F>` is `true`, and
- *`is-invocable-using`*`<F>` is `true`.

Preconditions: `f` is not a null pointer.

Effects: Initializes *`bound-entity`* with `f`, and *`thunk-ptr`* with the address of a function *`thunk`* such that *`thunk`*`(`*`bound-entity`*`, `*`call-args`*`...)` is expression-equivalent to `invoke_r<R>(f, `*`call-args`*`...)`.

```
template<class F> constexpr function_ref(F&& f) noexcept;
```

Let `T` be `remove_reference_t<F>`.

Constraints:

- `remove_cvref_t<F>` is not the same type as `function_ref`,
- `is_member_pointer_v<T>` is `false`, and
- *`is-invocable-using`*`<cv T&>` is `true`.

Effects: Initializes *`bound-entity`* with `addressof(f)`, and *`thunk-ptr`* with the address of a function *`thunk`* such that *`thunk`*`(`*`bound-entity`*`, `*`call-args`*`...)` is expression-equivalent to `invoke_r<R>(static_cast<`*`cv`*` T&>(f), `*`call-args`*`...)`.

```
template<auto f> constexpr function_ref(nontype_t<f>) noexcept;
```

Let `F` be `decltype(f)`.

Constraints: *`is-invocable-using`*`<F>` is `true`.

Mandates: If `is_pointer_v<F> || is_member_pointer_v<F>` is `true`, then `f != nullptr` is `true`.

Effects: Initializes *`bound-entity`* with a pointer to unspecified object or null pointer value, and *`thunk-ptr`* with the address of a function *`thunk`* such that *`thunk`*`(`*`bound-entity`*`, `*`call-args`*`...)` is expression-equivalent to `invoke_r<R>(f, `*`call-args`*`...)`.

```
template<auto f, class U>
  constexpr function_ref(nontype_t<f>, U&& obj) noexcept;
```

Let `T` be `remove_reference_t<U>` and `F` be `decltype(f)`.

Constraints:

- `is_rvalue_reference_v<U&&>` is `false`,
- *`is-invocable-using`*`<F, cv T&>` is `true`.

Mandates: If `is_pointer_v<F> || is_member_pointer_v<F>` is `true`, then `f != nullptr` is `true`.

Effects: Initializes *`bound-entity`* with `addressof(obj)`, and *`thunk-ptr`* with the address of a function *`thunk`* such that *`thunk`*`(`*`bound-entity`*`, `*`call-args`*`...)` is expression-equivalent to `invoke_r<R>(f, static_cast<`*`cv`*` T&>(obj), `*`call-args`*`...)`.

```
template<auto f, class T>
  constexpr function_ref(nontype_t<f>, cv T* obj) noexcept;
```

Let `F` be `decltype(f)`.

Constraints: *`is-invocable-using`*`<F, `*`cv`*` T*>` is `true`.

Mandates: If `is_pointer_v<F> || is_member_pointer_v<F>` is `true`, then `f != nullptr` is `true`.

Preconditions: If `is_member_pointer_v<F>` is `true`, `obj` is not a null pointer.

Effects: Initializes *`bound-entity`* with `obj`, and *`thunk-ptr`* with the address of a function *`thunk`* such that *`thunk`*`(`*`bound-entity`*`, `*`call-args`*`...)` is expression-equivalent to `invoke_r<R>(f, obj, `*`call-args`*`...)`.

```
template<class T> function_ref& operator=(T) = delete;
```

Constraints:

- `T` is not the same type as `function_ref`, and
- `is_pointer_v<T>` is `false`, and
- `T` is not a specialization of `nontype_t`.

Invocation

[func.wrap.ref.inv]

```
R operator()(ArgTypes... args) const noexcept(noex);
```

Effects: Equivalent to: `return thunk-ptr(bound-entity, std::forward<ArgTypes>(args)...);`

Deduction guides

[func.wrap.ref.deduct]

```
template<class F>
  function_ref(F*) -> function_ref<F>;
```

Constraints: `is_function_v<F>` is `true`.

```
template<auto f>
  function_ref(nontype_t<f>) -> function_ref<see below>;
```

Let `F` be `remove_pointer_t<decltype(f)>`.

Constraints: `is_function_v<F>` is `true`.

Remarks: The deduced type is `function_ref<F>`.

```
template<auto f, class T>
  function_ref(nontype_t<f>, T&&) -> function_ref<see below>;
```

Let `F` be `decltype(f)`.

Constraints:

- `F` is of the form `R(G::*)(A...)` *cv* `&`$_{opt}$ `noexcept(`*E*`)` for a type `G`, or
- `F` is of the form `M G::*` for a type `G` and an object type `M`, in which case let `R` be `invoke_result_t<F, T&>`, `A...` be an empty pack, and *E* be `false`, or
- `F` is of the form `R(*)(G, A...) noexcept(`*E*`)` for a type `G`.

Remarks: The deduced type is `function_ref<R(A...) noexcept(`*E*`)>`.

Feature test macro

Insert the following to [version.syn] (https://eel.is/c++draft/version.syn), header `<version>` synopsis:

```
#define __cpp_lib_function_ref 20XXXXL // also in <functional>
```

Implementation Experience

A complete implementation is available from ■ zhihaoy/nontype_functional@p0792r13 (https://github.com/zhihaoy/nontype_functional/tree/p0792r13).

Many facilities similar to `function_ref` exist and are widely used in large codebases. See Survey for details.

Qt 6 implemented R9 of this paper as ■ `qxp::function_ref` (https://codereview.qt-project.org/q/topic:function_ref).

Acknowledgments

Thanks to **Agustín Bergé**, **Dietmar Kühl**, **Eric Niebler**, **Tim van Deurzen**, **Alisdair Meredith**, and **Tim Song** for providing very valuable feedback on earlier drafts of this proposal.

Thanks to **Jens Maurer** for encouraging participation and **Tomasz Kamiński** for the thorough wording review.

References

1. *move_only_function* http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0288r9.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0288r9.html) ↩

2. *function_ref: a non-owning reference to a Callable* http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0792r5.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0792r5.html) ↩

3. The function_ref class template. *LLVM Programmer's Manual* https://llvm.org/docs/ProgrammersManual.html#the-function-ref-class-template (https://llvm.org/docs/ProgrammersManual.html#the-function-ref-class-template) ↩

4. *Implementing function_view is harder than you might think* http://foonathan.net/blog/2017/01/20/function-ref-implementation.html (http://foonathan.net/blog/2017/01/20/function-ref-implementation.html) ↩

5. *make function_ref more functional* https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2472r3.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2472r3.html) ↩